

category: avalonia

tags: [navigation, viewmodel-lifecycle, singleton, socket-server, rabbitmq, controller, reconnect]

severity: high

created: 2026-07-22

updated: 2026-07-22

project-origin: iPGSv4 (PAYMENT_KIOSK_VERTICAL)

NavigateTo tạo `new ViewModel(...)` mỗi lần → hạ tầng (socket/RabbitMQ/controller thiết bị) bị reconnect lại mỗi khi user quay về màn đó

Tình huống gặp phải

Kiosk app (Avalonia), màn `MainView` (frmMain gốc) là "idle screen" — user quay lại màn này liên tục sau MỖI giao dịch (tự động return hoặc bấm Back). `MainViewModel.OnLoadedAsync()` khởi động Socket Server (port 100), kết nối RabbitMQ, và Initialize Controller (thiết bị cửa/POS vật lý).

Triệu chứng / Lỗi

User báo: "mỗi lần quay lại main view lại phải chạy lại kết nối tới parking" — overlay "Connecting to parking server..." hiện lại mỗi lần về Main, dù app đã kết nối thành công từ trước.

Nguyên nhân gốc rễ (Root Cause)

`WindowNavigationService.NavigateToCore` (điều hướng kiểu swap `ContentControl.Content`, KHÔNG phải multi-window) tạo **instance ViewModel mới** ở mỗi case (`var vm = new MainViewModel(this)`) mỗi khi gọi `NavigateTo("Main")`. Đồng thời `MainView.OnDetachedFromVisualTree` gọi `_vm.OnUnloadedAsync()` — teardown TOÀN BỘ Socket Server/RabbitMQ connection/Controller subscription — mỗi khi user điều hướng SANG màn khác (rời Main).

Kết quả: attach → connect hạ tầng; detach (đi màn khác) → teardown hạ tầng; attach lại (quay về Main) → connect lại từ đầu. Vòng lặp này lặp lại ở MỖI giao dịch, dù hạ tầng (socket server, message broker, controller vật lý) về bản chất là tài nguyên **cấp app, không phải cấp màn hình** — WinForms gốc (`frmMain`) chỉ có 1 instance duy nhất sống suốt vòng đời app (ẩn/hiện, không bao giờ bị `new` lại), nên bug này không tồn tại ở bản gốc.

Đây là biến thể khác của lesson avalonia-onattachedtovisualtree-duplicate-event-wiring.md: ở đó là 1 UserControl bị attach lại NHIỀU LẦN cho CÙNG 1 instance (thiếu guard); ở đây là kiến trúc navigation tạo INSTANCE MỚI mỗi lần — nên fix không phải guard đơn thuần mà phải cache/singleton hoá ViewModel ở tầng NavigationService.

Giải pháp

1. Cache ViewModel làm singleton ở nơi tạo (NavigationService), thay vì `new` mỗi lần route tới:

```
```csharp
```

```
private MainViewModel? _mainViewModel;
case "Main":
 _mainViewModel ??= new MainViewModel(this);
 var view = new MainView(_mainViewModel);
 _host.MainContent.Content = view;
 break;
```

2. Guard method load hạ tầng bằng cờ instance (**IsLoaded**) để an toàn kép — dù ViewModel đã singleton, vẫn nên guard phòng trường hợp code gọi **OnLoadedAsync()** từ nơi khác:

```
```csharp
```

```
public bool IsLoaded { get; private set; }  
public async Task OnLoadedAsync()  
{  
    if (IsLoaded) return;  
    IsLoaded = true;  
    try { / connect socket/rabbitmq/controller / }  
    catch { IsLoaded = false; / cho phép retry nếu lỗi / }  
}
```

3. Bỏ lời gọi teardown hạ tầng khỏi lifecycle "rời màn hình" (**OnDetachedFromVisualTree**) — hạ tầng cấp app chỉ nên teardown khi THẬT SỰ cần (trước khi restart process để giải phóng port, hoặc app thoát) — KHÔNG phải mỗi lần user điều hướng sang màn khác rồi quay lại.
4. Nếu View có hiện loading overlay trong lúc connect, thêm guard **!vm.IsLoaded** trước khi show overlay — tránh overlay chớp nháy vô nghĩa ở lần attach thứ 2 trở đi.

Áp dụng lại (How to reuse)

- Khi review NavigationService kiểu "swap Content theo route key": bất kỳ route nào có logic connect hạ tầng (socket, message queue, TCP tới thiết bị vật lý, DB connection pool riêng...) trong **OnLoadedAsync/OnAttachedToVisualTree** → kiểm tra NGAY xem ViewModel có bị **new** lại mỗi lần route tới không.
- Dấu hiệu nhận biết: user báo "mỗi lần quay lại màn X lại phải chờ/kết nối lại", hoặc log cho thấy cùng 1 loại kết nối (RabbitMQ, controller) được tạo/dispose lặp đi lặp lại theo tần suất điều hướng thay vì tần suất khởi động app.
- Phân biệt 2 loại ViewModel khi thiết kế navigation:
 - **Cấp app** (Main/idle screen, service hạ tầng) → PHẢI singleton, cache theo route key.
 - **Cấp giao dịch** (Detail xe, thanh toán theo biển số...) → tạo mới là ĐÚNG vì dữ liệu phụ thuộc tham số giao dịch (biển số, phí tính theo thời gian) — KHÔNG cache/reuse các màn này chỉ vì lo tối ưu, vì cache sai sẽ hiển thị phí/dữ liệu cũ (rủi ro nghiêm trọng hơn cho kiosk thanh toán).

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Sau khi singleton hoá ViewModel, PHẢI bỏ luôn lời gọi teardown ở `OnDetachedFromVisualTree` — nếu chỉ thêm guard `IsLoaded` mà vẫn giữ teardown khi detach, hạ tầng sẽ bị stop lần đầu rời màn và KHÔNG BAO GIỜ reconnect lại (vì `IsLoaded` đã true nên `OnLoadedAsync` bỏ qua) → lỗi nặng hơn bug gốc.
- ⚠ Đừng áp dụng "singleton hoá" tràn lan cho MỌI ViewModel trong navigation stack — chỉ áp dụng cho màn hình đóng vai trò hạ tầng/idle (Main). Các màn giao dịch theo tham số (CarLocation, CarInforDetail, PaymentInfo...) phải tiếp tục tạo mới + gọi lại API mỗi lần vào, vì dữ liệu hiển thị (đặc biệt là số tiền/phí) phải luôn mới — cache sai ở đây là lỗi nghiêm trọng hơn việc gọi API thừa.
- ⚠ Khi thêm cờ `IsLoaded/_isLoaded`, KHÔNG dùng `static` (sẽ share nhầm giữa các instance nếu sau này lại có nhiều instance) — chỉ đúng vì đã đảm bảo route đó luôn dùng đúng 1 instance qua NavigationService cache.

Tham chiếu

- File: `IPGS.Kiosk.Avalonia/Views/MainWindow.axaml.cs`,
`IPGS.Kiosk.Avalonia/ViewModels/MainViewModel.cs`,
`IPGS.Kiosk.Avalonia/Views/MainView.axaml.cs`
- Project liên quan: iPGSv4 (kiosk thanh toán, nhánh PAYMENT_KIOSK_VERTICAL)
- Lesson liên quan: avalonia-onattachedtovisualtree-duplicate-event-wiring.md, avalonia-goback-history-reenters-autonaviate-waiting-screen.md